

Integration and Evaluation of Different FPGA based Hardware Accelerators for the AES Algorithm in the TLS Protocol to be Used in Critical Automation Systems

Horn, Luca; Frauenschläger, Tobias; Mottok, Jürgen
Ostbayerische Technische Hochschule Regensburg

Abstract

This thesis addresses the challenges of hardware-assisted cryptography in the context of automation systems in critical infrastructures. While hardware accelerators enhance cryptographic calculations in both performance and security, their permanence poses issues in adapting to evolving security concerns. To overcome this problem, we integrated and evaluated a hardware accelerator for the AES algorithm on an FPGA within a TLS application, comparing its performance with other accelerators. Employing a hybrid FPGA-CPU processor system, this setup achieves over 800 Mbit/s data rates, showcasing a more than 27 times acceleration in TLS transmissions compared to purely software-based solutions. The FPGA's flexibility and upgradability therefore make it an ideal platform for critical infrastructure, ensuring longevity and superior acceleration in TLS data transfers.

1. Introduction

In response to the increasing threat of cyberattacks on data communication in critical infrastructures in Germany [1], the Federal Office for Information Security (BSI) enacted the IT Security Act 2.0 ("IT-Sicherheitsgesetz 2.0") mandating secure IT system operation for critical infrastructure operators [2]. However, a challenge persists in certifying critical infrastructure hardware for 15 - 30 years while ensuring the security measures align with evolving standards. This work addresses this issue by integrating an accelerator for the Advanced Encryption Standard (AES) algorithm in Galois/Counter Mode (GCM) on a hybrid system with an FPGA and an ARM processor. Multiple accelerator versions are thereby integrated to accommodate for different scaling levels. The hardware's adaptability could minimize the need for hardware replacement in the face of security vulnerabilities or compromised algorithms if sufficient data rates can be achieved. After presenting our design, data rates in the widely used TLS protocol are evaluated to get an indication of the performance achievable in a real-world scenario.

2. Related Work

To understand this work, a basic understanding of hardware-software co-design is needed, including the functionality of FPGAs and especially Lookup Tables (LUTs) [3]. In addition, an understanding of the various AXI buses and of DMA controllers is expected. Finally, some cryptographic knowledge is required, in particular the functionality of the AES algorithm in GCM mode [4] and of TLS.

The following sections first cover accelerators implemented on custom ASICs, followed by accelerators implemented on FPGAs.

2.1. AES accelerators as custom ASICs

One way to speed up the AES algorithm is to provide the generic processor with special instructions that accelerate specific operations of the algorithm, e.g. AES New Instructions (AES-NI) peripherals in x86 processors. In a benchmark, AES-NI is evaluated by [5], showing that a performance gain of about 12 times can be achieved with AES-NI. However, it should be noted that AES-NI is only available for the x86 architecture, and thus, the speeds of 2900 MB/s are achieved with processors which normally are not used in the embedded context. Embedded devices, on the other hand, usually use less powerful processors such as ARM processors, which is why the speed gain from adding AES instruction sets must be interpreted with caution in the context of this work.

As an industrial solution, the cryptographic accelerator within the STM32H7xx processors, which are based on a 32-bit ARM architecture, will be considered. Of particular interest thereby is how the data to be encrypted is made available to the accelerator peripheral [6]. To start the encryption or decryption process, the key and the Initialisation Vector (IV) must first be written to corresponding registers. Then, a DMA is used to transfer data between the processor and the accelerator. Finally, the authentication tag can be read from a corresponding register. The CycloneCRYPTO benchmark was used in [7] to evaluate the data sheet's claims of 656 Mbit/s in bandwidth [6]. In the benchmark, a lower bandwidth of 360.36 Mbit/s was achieved. Despite not reaching exceptionally high speeds, the acceleration achieved compared to software-based computations is substantial. As demonstrated by [7], the software implementation lags significantly behind in terms of speed with a maximum data rate of 32.28 Mbit/s, representing an 11 times decrease in bandwidth.

2.2. AES accelerators implemented on FPGAs

In [8], it is shown that an implementation in a hardware description language is capable of achieving a high bandwidth. By using pipelining, a bandwidth of 33.66 Gbit/s at a clock rate of 263 MHz could be reached for AES-GCM with a key length of 256 bit on a Virtex-4LX-60ff668-11 board from Xilinx. The resource utilization thereby was 19301 slices. By using BRAM, the resource utilization could be reduced to about half of the

slices, with the maximum frequency dropping to 243 MHz and the bandwidth to 31.10 Gbit/s. The performance tests were repeated on the Virtex-5LX85ff668-2 board from Xilinx. There, a bandwidth of 39.04 Gbit/s could be achieved with a maximum frequency of 305 MHz and a resource utilization of 8077 slices, showcasing that great acceleration for AES can be reached on FPGAs.

3. Reference design

The hardware design presented in this work can encrypt and decrypt data using a hardware implementation of the AES-GCM algorithm. The data for the AES accelerator is provided by a DMA from the connected processor. To ease software integration, a multi-layer driver was developed.

As an FPGA, the Digilent Arty Z7-20 was used. The board is equipped with a Gigabit-Ethernet-PHY and an RJ45 connector for network connections. The dual-core ARM-Cortex A9 processor runs at a speed of 650 MHz, while the FPGA runs at a clock frequency of 100 MHz.

For the hardware architecture, a block design was created that contains various IP-cores, shown schematically in Fig. 1. The central control is done by the processor, which is integrated on the development board. The AES core is an open-source IP-core provided by Luca Berghella [9]. The core is highly customizable and can operate with a key length of 128, 192 or 256 bit. For this work, a key length of 256 bit was chosen. The Linear Feedback Shift Register (LFSR) is used by the processor and the cryptographic library to create pseudo random data.

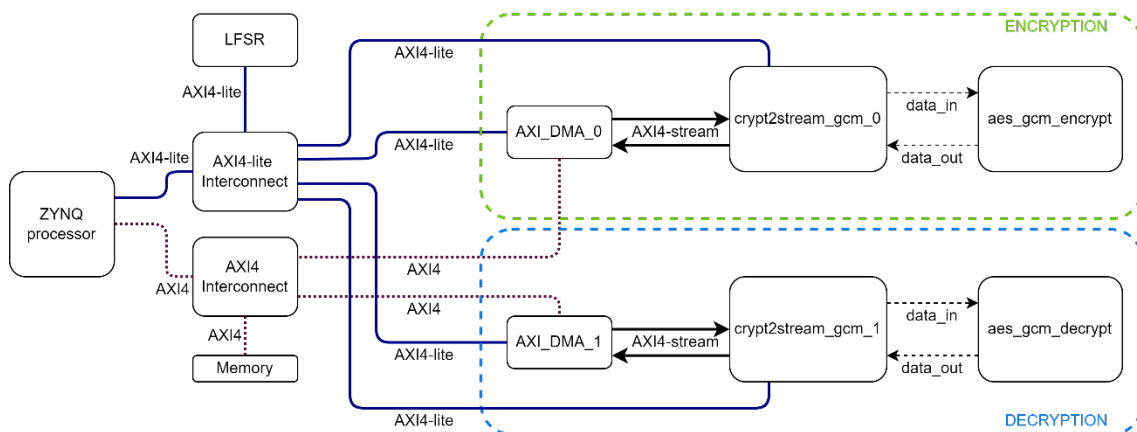


Fig. 1: Schematic representation of the hardware architecture implemented on the FPGA, visually separated into encryption and decryption.

The concept for providing data to the AES core is based upon [6], which was already covered in Section 2.1. For encryption, the data is read from a previously reserved

memory area of the DDR3 Random Access Memory (RAM) using a DMA, which provides that data to the AES core via an AXI-stream bus. Over the AXI4-lite bus, other required data like the IV or the key is provided. Since the AES core does not have an AXI-stream interface, another IP core named *crypt2stream_gcm* splits the stream into several 128 bit vectors to match the AES block size [4]. These blocks are gradually made available to the AES hardware accelerator (*aes_gcm_encrypt*). After the encryption is complete, the encrypted 128 bit vectors are converted back into an AXI-stream using the *crypt2stream_gcm* component and written to a second reserved memory area by the DMA controller. As soon as all data is encrypted and written back into memory, the DMA triggers an interrupt, which is connected to the processor's interrupt controller. After the interrupt is recognized, the data can be read by the processor. The decryption process is analogue. The following sections first go into detail on how the AES core works. Subsequently, the *crypt2stream_gcm* component is explained in detail.

3.1. AES-GCM Core

This section is based on Luca Berghella's GitHub repository [9], featuring the AES-GCM IP-core used in this work. The IP-core is highly customizable, allowing configuration of key size, pipelining level, overall size, and speed of the AES core. The IP-core comes in four sizes: XS, S, M, and L, each with varying resource usage impacting speed. The largest core achieves a theoretical bandwidth of over 12 Gbit/s at 100 MHz, while the XS core achieves near gigabit speeds. Scaling the frequency, the core matches speeds of solutions in related work (Section 2.2), indicating a high acceleration for AES-GCM. Actual bandwidths will be assessed with benchmarks in Section 4. Detailed configuration information is available in the GitHub repository, containing information on how the different sizes and speeds are achieved.

3.2. crypt2stream controller

The *crypt2stream_gcm* IP-core is responsible for reading all relevant data from the processor, encrypting or decrypting it using the AES-GCM core, and passing the processed data, as well as the authentication tag, back to the processor. Fig. 2 shows a schematic of the hardware architecture of the IP-core.

The component has four main functions, which are coloured in four different colours in the figure and explained in the following.

Register-based data transmission (green): The green blocks are responsible for the register-based data transfer via the AXI4-lite bus, over which the core is controlled.

Receiving data from the DMA (yellow): The data to be encrypted or decrypted is transferred from the processor to the *crypt2stream_gcm* component with the help of a DMA. The DMA outputs the data read from memory via an AXI4-stream bus. The AXI4-

stream outputs a data size of 32 bit, but the AES algorithm expects a data size of 128 bit per block. For this reason, four 32 bit blocks of data are buffered and then written into a 128 bit wide FIFO. The 128 bit data blocks can then be read by the *AES_GCM_CONTROLLER* component from the input FIFO.

Sending data to the DMA (blue): The processed data is written back to memory again using a DMA. The entire logic for sending the data to the DMA is shown in the colour blue. To prevent any data loss, the 128 bit vectors are first written back into a FIFO. From there, they are then sent back to the DMA as four 32 bit blocks.

Controlling the AES-GCM core (red): After the other three groups prepare all data for the correct size and endianness, the AES-GCM IP-core can be fed with this data. This is represented by the component *AES_GCM_CONTROLLER* in the colour red. The AES-GCM core is controlled by a state machine, which operates according to the descriptions in [9].

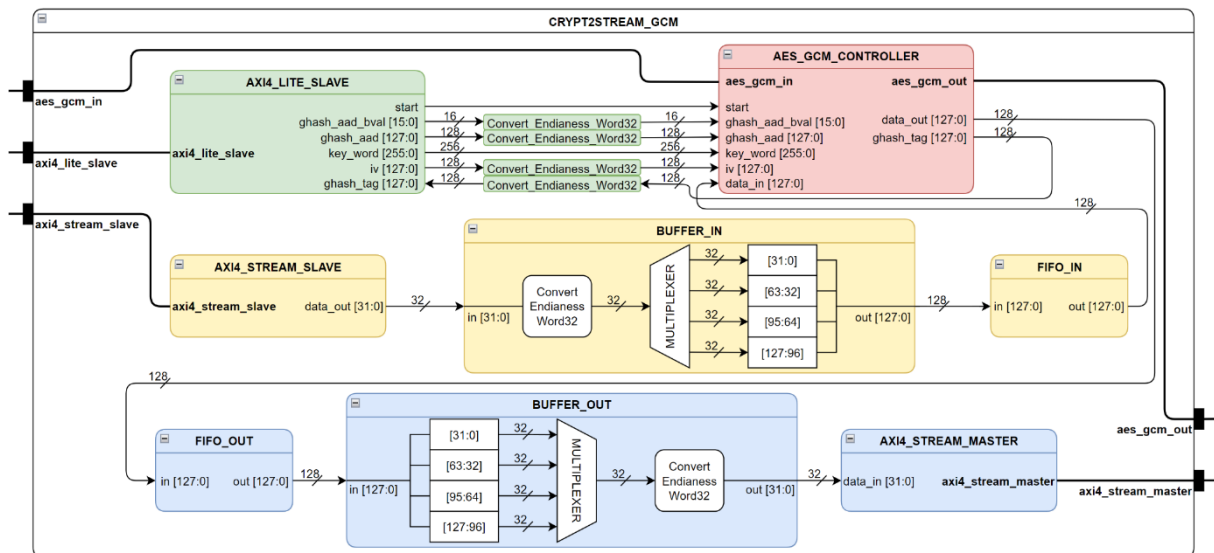


Fig. 2: Hardware architecture of the *crypt2stream_gcm* IP-core. The thick lines are several signals grouped by category. The thin lines represent individual signals, whereby the numbers on the line show the size of the vectors in bits.

4. Benchmarks

To test the functionality and performance of the hardware acceleration, benchmarks were carried out. The following section first provides information on the used AES-GCM IP-core variants. Then the performance of the different variants of the AES-GCM IP-core is evaluated for encryption and decryption. Subsequently, the achievable bandwidth when transmitting data with TLS is measured when using different variants of the cryptographic accelerator. Finally, we compare the TLS bandwidth of a software-only implementation with that of the cryptographic hardware accelerator.

In total, four different variants of the AES-GCM IP-core were synthesized and tested for both encryption and decryption using the following naming scheme:

<algorithm>_<encryption/decryption>_<key_length>_<pipe_stages>_<size>

When the encryption/decryption field is missing, this means that no single IP-core is meant, but a whole system with an encryption and decryption core including all the necessary peripherals is referred to. In total, the three sizes XS, S and M were synthesized with a pipe stage of 0. In addition, the size S was synthesized with a pipe stage of 2 to see the impact on timing constraints as well as latency when using pipelining. The size L was not synthesized, as no speed increase can occur when stepping up from the M variant in our case. This is due to the minimal latency of the *crypt2stream_gcm* component, limiting the maximum speed of the overall system.

4.1. Encryption speed with different accelerators

To evaluate the performance of the hardware encryption, a speed test was run. The encryption speed was measured for single blocks with the sizes 4, 16, 32, 64, 256, 1024, 4096, 16384 and 65536 bytes. For every data length, a new random key as well as a new IV were generated using the LFSR. For each data length, 128 bit of AAD was provided. The benchmarks were run for all four AES core variants. The results can be seen in Fig. 3.

As can be seen in the plot, the encryption speed is strongly dependent on the data size. This is due to the overall system adding a certain time overhead to the actual encryption process. On the one hand, the DMA adds a certain overhead with the data transfer. Also, filling the registers with data takes some time. However, this time remains constant and is only significant for smaller data sizes of up to 1 kB, which can be seen as the time needed for encryption does not change at all. The only reason that the data rate increases is that the same encryption time is needed for a larger amount of data. It can be concluded from this that up to a data size of 1 kB, the time overhead introduced by the bus systems far outweighs the actual encryption time.

The encryption times start to increase above a data size of 1 kB, where differences between the IP-core variants can also be observed. For example, the XS core is on average 22 - 30% slower than the other three variants, which all achieve a similar data rate of up to 800 Mbit/s. The M core is about 3% faster than the S core without pipelining at the maximum data size of 64 kB. However, this small increase in speed is probably due to limitations of the *crypt2stream_gcm* component. Although the use of pipelining in the S core does not change the data rate, it increases the latency. The differences in data rate are therefore likely to be due to this effect.

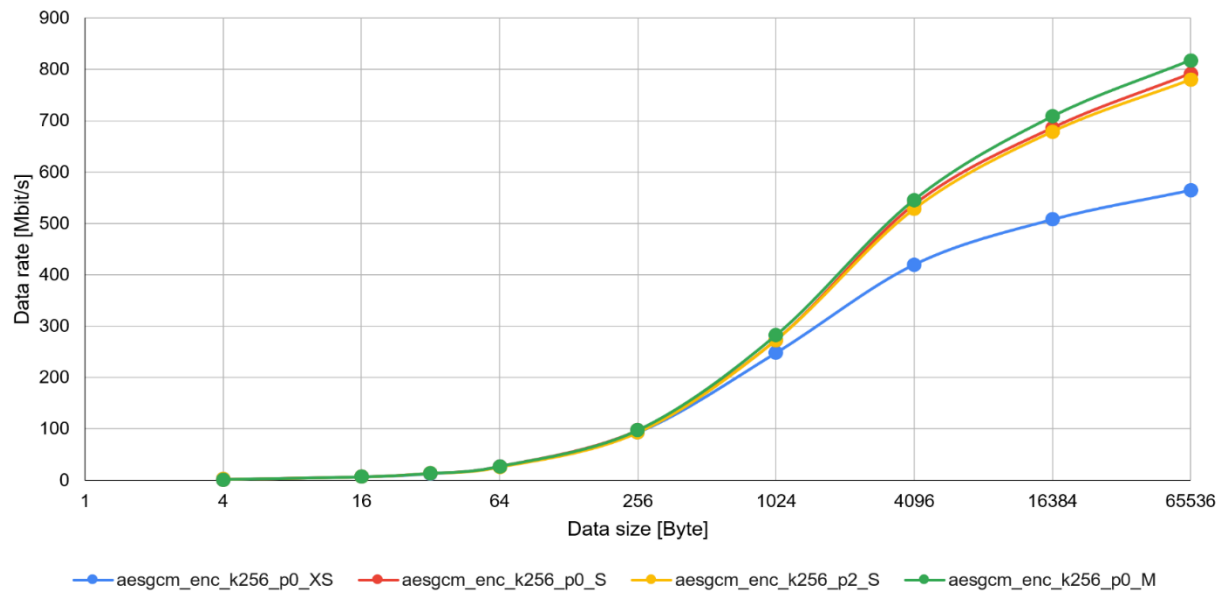


Fig. 3: Encryption data rate of different AES-GCM cores for different data sizes.

Finally, the achieved data rates in encryption are to be compared with the resource consumption in LUTs. This is shown graphically in Fig. 4. Here, the data rate for the encryption of 64 kB of data is plotted for all four variants of the AES-GCM core over their LUT consumption. The graph clearly shows that the XS variant has almost the same resource usage as the S variant with pipelining but is 28% slower. Both S variants achieve a similar data rate, whereby the variant without pipelining is slightly larger than the one with pipelining but offers less latency. With a data rate that is about 3% higher than that of the S core, the M variant offers only a marginal increase in speed compared to the 39% more resources that are required. This suggests that the XS and the M core are not viable with the system's current constraints. The most sensible choice is therefore the S-sized AES-GCM core. Whether pipelining should be used depends on whether a safer timing and smaller size (with pipelining) or a lower latency (without pipelining) is preferred.

Due to the way AES works in GCM mode, the same data rates are achieved for decryption as for encryption. The decryption benchmarks are therefore not discussed in this work. The benchmarks show that good data rates can be achieved with hardware acceleration on an FPGA. Compared to the accelerator of the STM32 microcontrollers, more than twice the data rate is achieved with the FPGA despite a lower clock rate (see Section 2.1).

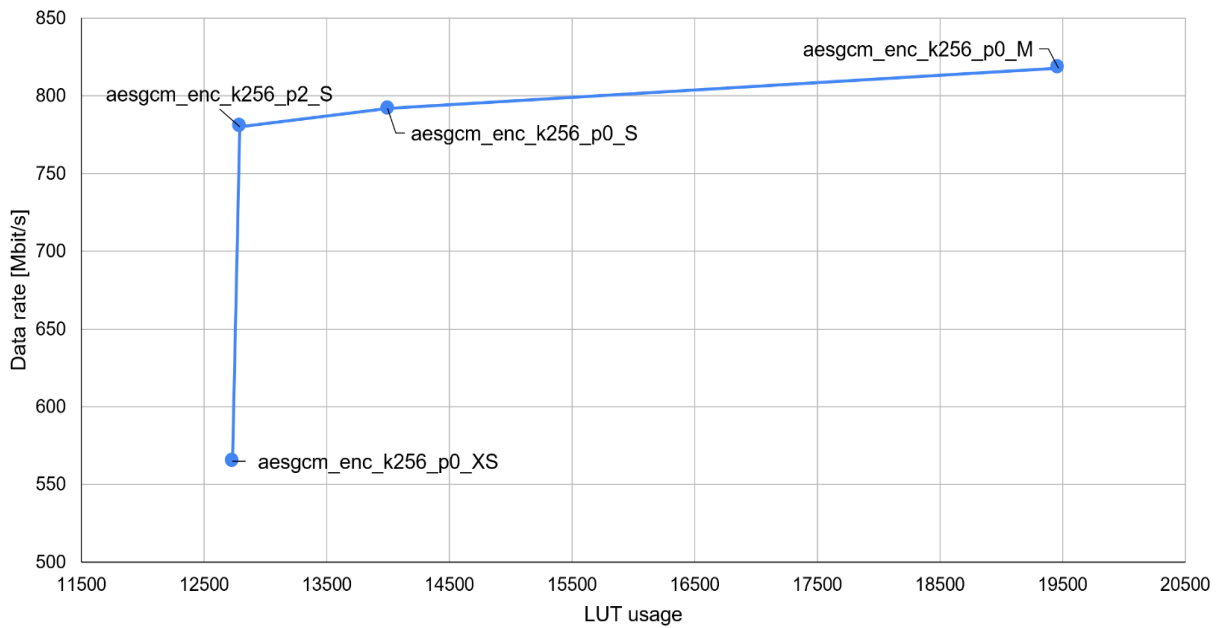


Fig. 4: Encryption data rates over LUT usage for all four IP-core variants with a data size of 64 kB.

4.2. TLS performance comparison

To measure the transmission speed, it was decided to use the well-known *iperf* test, which can be used to measure the bandwidth of a TCP connection between a client and a server [10]. The test was run unidirectional with the FPGA as the server. The FPGA is thereby responsible for the decryption and the time measurement. Since the *iperf* test naturally only supports TCP, it was adapted for the TLS performance measurements. All Ethernet connections in the test were Gigabit-Ethernet connections, allowing for a theoretical maximum data rate of 1000 Mbit/s. The results of the modified *iperf* test over TLS are shown in Tab. 1.

Setup variant	iperf (2892 Byte Payload)		
	Time [s]	Transfer [MB]	Bandwidth [Mbit/s]
aesgcm_k256_p0_XS	21,2	302	120
aesgcm_k256_p2_S	21,2	318	126
WolfSSL Software	24,6	13,5	4,6
TCP	30,0	621	173

Tab. 1: Achievable data rates with FPGA hardware acceleration.

As the M core and the two S variants achieve nearly the same speeds at a data size of 2892 Byte, only the XS variant and the S variant with pipelining were included in the

test. For the pure software setup without AES acceleration, the implementation provided by the TLS library (WolfSSL) has been used. In addition, the iperf test was also run using TCP to obtain a reference value. Next to the achieved bandwidth, the transmission time in seconds as well as the amount of transferred data in MB is shown.

With the AES-GCM cores in the XS and S variants, a bandwidth with TLS of 120 and 126 Mbit/s respectively is achieved. While the XS core was already about 50 Mbit/s slower than the S variant at this data size in the benchmarks, only a difference of 6 Mbit/s can be observed in the iperf test. Basically, however, both IP-core variants are far below the more than 400 Mbit/s that were achieved by both core variants for the same data size in the benchmarks. In a TCP transmission without encryption, a data rate of 173 Mbit/s could be achieved. Thus, with activated encryption and hardware acceleration, about 26% of bandwidth is lost. Since the bandwidth with TCP is still far below the benchmark data rates of the two IP-core variants and the XS and S cores hardly differ in data rate, it can be concluded that the TLS bandwidth is strongly limited by the network driver.

While 126 Mbit/s are achieved with hardware acceleration, only a data rate of 4.6 Mbit/s is achieved with a pure software implementation. This corresponds to a more than 27-fold acceleration of the data rate. Thus, hardware acceleration on the FPGA achieves more than twice the acceleration achieved with AES-NI or the STM32 cryptographic accelerator (see Section 2).

5. Conclusion and outlook

High data rates in both encryption and decryption are achieved with the overall system. This is especially reflected in the acceleration achieved in data transfers with the TLS protocol, where a 27 times higher data rate is achieved with the accelerator than with a pure software implementation. This exceeds the speed increase of conventional hardware accelerators from related work by at least two times. It can therefore be concluded that an FPGA is suitable for accelerating the AES-GCM algorithm. Especially for critical infrastructure, this is an excellent approach to meet the requirements of the IT Security Act 2.0 while ensuring the longevity of the hardware and the IT security of the installed devices.

References

- [1] BSI: Die Lage der IT-Sicherheit in Deutschland. Online, accessed on 10.03.2024, <https://www.bsi.bund.de/SharedDocs/Downloads/DE/BSI/Publikationen/Lageberichte/Lagebericht2022.pdf>.
- [2] BSI: IT-Sicherheitsgesetz 2.0. Bundesgesetzblatt Jahrgang 2021 Teil I, Nr. 25. Online, accessed on 10.03.2024, www.bundesanzeiger-verlag.de.

- [3] Kennings, Andrew et al.: Efficient FPGA Resynthesis Using Precomputed LUT Structures. 2010 International Conference on Field Programmable Logic and Applications (FPL 2010), Milan, Italy, 31.08.2010 - 02.09.2021, pp. 532–537.
- [4] McGrew, David; Viega, John: The Galois/Counter Mode of Operation (GCM). Online, accessed on 10.03.2024, <https://csrc.nist.gov/groups/ST/toolkit/BCM/documents/proposedmodes/gcm/gcm-spec.pdf>.
- [5] Shawn, Wilkinson: An Analysis of the AES-NI Extension to the x 86 Instruction Set. Online, accessed on 10.03.2024, <https://www.semanticscholar.org/paper/An-Analysis-of-the-AES-NI-Extension-to-the-x-86-Set-Wilkinson/b9783174b83d8af25ce9034cd636b0b0d97b31ef>.
- [6] STMICROELECTRONICS: STM32H7xx Reference manual. Online, accessed on 10.03.2024, https://www.st.com/resource/en/reference_manual/dm00314099-stm32h742-stm32h743-753-and-stm32h750-value-line-advanced-arm-based-32-bit-mcus-stmicroelectronics.pdf.
- [7] Oryx, Embedded: STM32H7 Crypto Benchmark. Online, accessed on 10.03.2024 <https://www.oryx-embedded.com/benchmark/st/crypto-stm32h7.html>.
- [8] Zhou, Gang et al.: Improving Throughput of AES-GCM with Pipelined Karatsuba Multipliers on FPGAs. Reconfigurable Computing: Architectures, Tools and Applications: 5th International Workshop, ARC 2009, Karlsruhe, Germany, 16.03.2009 - 18.03.2009, pp. 193–203.
- [9] Berghella, Luca: Configurable AES-GCM IP. Online, accessed on 10.03.2024, <https://github.com/BLu85/AES-GCM-128-192-256-bits>.
- [10] Tolly, Kevin: Mit iPerf3 die Netzwerkbandbreite testen. Online, accessed on 10.03.2024, <https://www.computerweekly.com/de/tipp/Wie-man-mit-iPerf3-die-Netzwerkbandbreite-testet>.

Kontakt

Tobias Frauenschläger, M.Sc.
Ostbayerische Technische Hochschule Regensburg
Seybothstraße 2
93053 Regensburg
E-Mail: tobias.frauenschlaeger@oth-regensburg.de